

Exercise 1: Code Understanding Journal

Application: Java Task Manager

Architecture

The application follows a layered architecture: CLI (TaskManagerCli) -> TaskManager -> TaskStorage -> tasks.json. The CLI parses commands, TaskManager implements business logic, TaskStorage persists tasks using Gson JSON serialization, and the model layer contains Task, TaskStatus and TaskPriority.

Task Creation Flow

1. User runs the create command.
2. TaskManagerCli validates arguments and calls createTask().
3. TaskManager converts the priority and due date.
4. A Task object is created with a UUID, timestamps and TODO status.
5. TaskStorage.addTask() stores it in memory and saves it to tasks.json.

Status Update Flow

The CLI calls updateTaskStatus(). The TaskManager retrieves the task from storage, updates its status, marks completion time if DONE, and saves the updated collection.

Priority System

Priorities are represented by the TaskPriority enum (LOW, MEDIUM, HIGH, URGENT). Filtering and statistics use this enum rather than raw integers.

Data Persistence

Tasks are stored in a HashMap and serialized to tasks.json using Gson with custom LocalDateTime serializers.

Design Patterns

- Layered Architecture
- Separation of Concerns
- Repository-style storage layer
- Domain Model (Task)

Reflection

Using AI to inspect the code helped identify the execution flow without modifying the application. The storage layer, business logic and CLI are cleanly separated, making the code easier to understand and maintain.